Proving Data Structures Correct: Balanced Search Trees

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Insertion: The Idea

©Mohammad Abdulaziz

Insertion: The Idea

`Leaf` $\rightsquigarrow$ `Node2`

©Mohammad Abdulaziz

Insertion: The Idea

```
Leaf   ⇝   Node2
Node2  ⇝   Node3
```

©Mohammad Abdulaziz

Insertion: The Idea

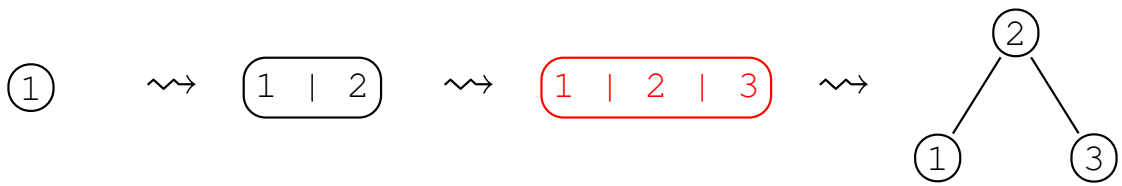
`Leaf`	$\rightsquigarrow$	`Node2`
`Node2`	$\rightsquigarrow$	`Node3`
`Node3`	$\rightsquigarrow$	overflow, pass 1 element back up

©Mohammad Abdulaziz

Insertion: The Idea

`Leaf`	$\rightsquigarrow$	`Node2`
`Node2`	$\rightsquigarrow$	`Node3`
`Node3`	$\rightsquigarrow$	overflow, pass 1 element back up

Inserting 1, 2, 3:

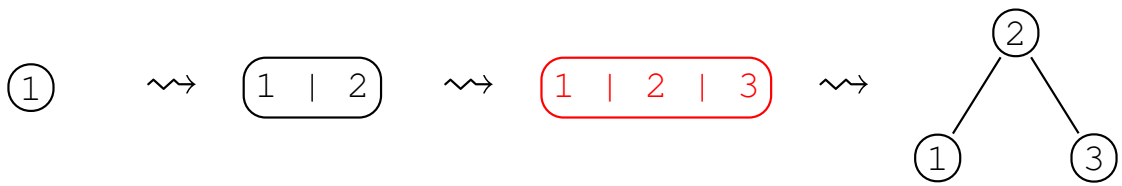


©Mohammad Abdulaziz

Insertion: The Idea

`Leaf`	$\rightsquigarrow$	`Node2`
`Node2`	$\rightsquigarrow$	`Node3`
`Node3`	$\rightsquigarrow$	overflow, pass 1 element back up

Inserting 1, 2, 3:



The tree grows at the root: all leaves stay at the same level

©Mohammad Abdulaziz

Insertion: Two Possible Results

`ins x t` either

©Mohammad Abdulaziz

Insertion: Two Possible Results

`ins x t` either

- accommodates `x` without increasing the height: $\texttt{Eq}_i\ \texttt{t'}$

Insertion: Two Possible Results

`ins x t` either

- accommodates `x` without increasing the height: $\texttt{Eq}_i$ `t'`
- overflows: `Of l a r`

©Mohammad Abdulaziz

Insertion: Two Possible Results

`ins x t` either

- accommodates `x` without increasing the height: `Eq`$_i$ `t'`
- overflows: `Of l a r`

```
datatype 'a up_i = Eq_i ('a tree23) | Of ('a tree23) 'a ('a tree23)
```

Insertion: Two Possible Results

`ins x t` either

- accommodates `x` without increasing the height: `Eq`$_i$ `t'`
- overflows: `Of l a r`

```
datatype 'a up_i = Eq_i ('a tree23) | Of ('a tree23) 'a ('a tree23)

tree_i :: 'a up_i ⇒ 'a tree23
tree_i (Eq_i t) = t
tree_i (Of l a r) = ⟨l, a, r⟩
```

Insertion: Two Possible Results

`ins x t` either

- accommodates `x` without increasing the height: `Eq`$_i$ `t'`
- overflows: `Of l a r`

```
datatype 'a up_i = Eq_i ('a tree23) | Of ('a tree23) 'a ('a tree23)

tree_i :: 'a up_i ⇒ 'a tree23
tree_i (Eq_i t) = t
tree_i (Of l a r) = ⟨l, a, r⟩

insert x t = tree_i (ins x t)
ins :: 'a ⇒ 'a tree23 ⇒ 'a up_i
```

Insertion: `Leaf` and `Node2`

```
ins x Leaf = Of Leaf x Leaf
```

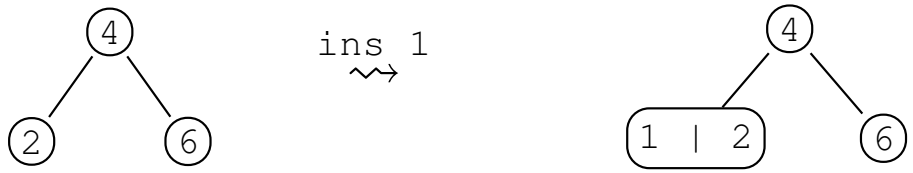
©Mohammad Abdulaziz

Insertion: `Leaf` and `Node2`

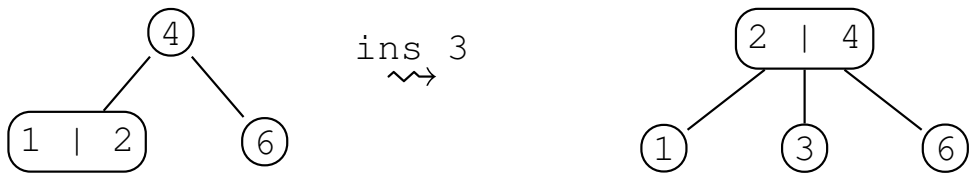
```
ins x Leaf = Of Leaf x Leaf

ins x ⟨l, a, r⟩ =
  (if x < a then (case ins x l of
                    Eqi l' ⇒ Eqi ⟨l', a, r⟩
                  | Of l1 b l2 ⇒ Eqi ⟨l1, b, l2, a, r⟩) else
   if x > a then (case ins x r of
                    Eqi r' ⇒ Eqi ⟨l, a, r'⟩
                  | Of r1 b r2 ⇒ Eqi ⟨l, a, r1, b, r2⟩)
   else Eqi ⟨l, a, r⟩)
```

Insertion into `Node2`: Examples



`Leaf` overflows; the 2-node `2` absorbs `1` and becomes a 3-node: $\texttt{Eq}_i$



`1 | 2` overflows: `Of` ⟨`1`⟩ `2` ⟨`3`⟩; the 2-node `4` absorbs `2`: $\texttt{Eq}_i$

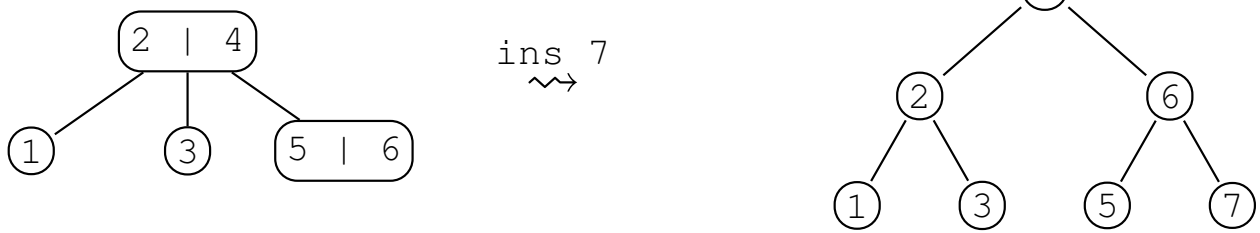
©Mohammad Abdulaziz

Insertion: Node3

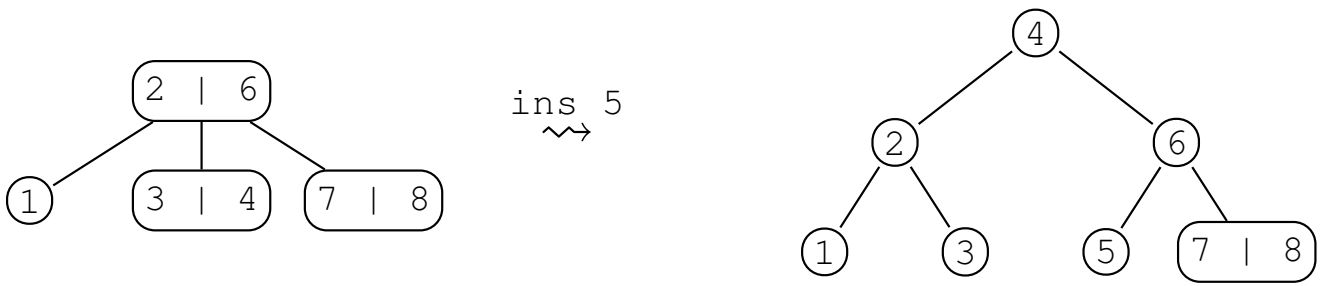
```
ins x ⟨l, a, m, b, r⟩ =
  (if x < a then (case ins x l of
                    Eq_i l' ⇒ Eq_i ⟨l', a, m, b, r⟩
                  | Of l1 c l2 ⇒ Of ⟨l1, c, l2⟩ a ⟨m, b, r⟩) else
   if x = a then Eq_i ⟨l, a, m, b, r⟩ else
   if x < b then (case ins x m of
                    Eq_i m' ⇒ Eq_i ⟨l, a, m', b, r⟩
                  | Of m1 c m2 ⇒ Of ⟨l, a, m1⟩ c ⟨m2, b, r⟩) else
   if x = b then Eq_i ⟨l, a, m, b, r⟩
   else (case ins x r of
           Eq_i r' ⇒ Eq_i ⟨l, a, m, b, r'⟩
         | Of r1 c r2 ⇒ Of ⟨l, a, m⟩ b ⟨r1, c, r2⟩))
```

©Mohammad Abdulaziz

Insertion into `Node3`: Examples



`5 | 6` overflows, then `2 | 4` overflows: `insert` makes `4` the new root, the height grows



middle case: `3 | 4` overflows: `Of ⟨3⟩ 4 ⟨5⟩`; `4` moves up, `2 | 6` splits around it

Insertion Preserves `complete`

Lemma

$$\texttt{complete t} \implies \texttt{complete (tree}_i\texttt{ (ins a t))} \wedge \texttt{h}_i\texttt{(ins a t)} = \texttt{h(t)}$$

©Mohammad Abdulaziz

Insertion Preserves `complete`

Lemma

$$\texttt{complete t} \implies \texttt{complete (tree}_i\texttt{ (ins a t))} \wedge \texttt{h}_i\texttt{(ins a t)} = \texttt{h(t)}$$

Proof by induction on `t`.

Insertion Preserves `complete`

Lemma

`complete t` $\Longrightarrow$
`  complete (tree`$_i$ `(ins a t))` $\wedge$ `h`$_i$`(ins a t) = h(t)`

where `h`$_i$ `::  'a up`$_i$ $\Rightarrow$ `nat`

`h`$_i$ `(Eq`$_i$ `t) = h(t)`

`h`$_i$ `(Of l a r) = h(l)`

Proof by induction on `t`.

©Mohammad Abdulaziz

Insertion Preserves `complete`

Lemma

`complete t` $\Longrightarrow$
`  complete (tree`$_i$ `(ins a t))` $\wedge$ `h`$_i$`(ins a t) = h(t)`

where `h`$_i$ `::  'a up`$_i$ $\Rightarrow$ `nat`

`h`$_i$ `(Eq`$_i$ `t) = h(t)`

`h`$_i$ `(Of l a r) = h(l)`

Proof by induction on `t`. Base and step automatic.

©Mohammad Abdulaziz

Insertion Preserves `complete`

Lemma

```
complete t ⟹
  complete (tree_i (ins a t)) ∧ h_i(ins a t) = h(t)
```

where `h_i :: 'a up_i ⇒ nat`

```
h_i (Eq_i t) = h(t)
h_i (Of l a r) = h(l)
```

Proof by induction on `t`. Base and step automatic.

Corollary

```
complete t ⟹ complete (insert a t)
```

©Mohammad Abdulaziz

Beyond 2-3 Trees

©Mohammad Abdulaziz

Beyond 2-3 Trees

Deletion: `Node2` $\rightsquigarrow$ underflow, merge with siblings on the way up

Beyond 2-3 Trees

Deletion: `Node2` $\rightsquigarrow$ underflow, merge with siblings on the way up

datatype `'a tree234 =`
`  Leaf | Node2 ... | Node3 ... | Node4 ...`

Like 2-3 trees, but with more cases

©Mohammad Abdulaziz

Beyond 2-3 Trees

Deletion: `Node2` $\rightsquigarrow$ underflow, merge with siblings on the way up

datatype `'a tree234 =`
`  Leaf | Node2 ... | Node3 ... | Node4 ...`

Like 2-3 trees, but with more cases

The general case:

B-trees and (a, b)-trees